Universitatea Națională de Știință și Tehnologie Politehnica București

Facultatea de Științe Aplicate

Proiect
Programare Liniară

An III – Grupa 1333a

Disciplina: Proiect Cercetări Operaționale

2026

Echipa de proiect:

Grupa 1333a

Dedu Anișoara-Nicoleta

Dumitrescu Andreea-Mihaela

Iliescu Daria-Gabriela

Lungu Ionela-Diana

Cuprins

INTRODUCERE **3**

CERCETAREA OPERAȚIONALĂ ȘI ALGORITMUL SIMPLEX 3

1. Geneza Cercetării Operaționale (CO) *3*

2. George Dantzig: „Creierul" din spatele Programării Liniare *3*

3. Mecanismul Simplex: Logică și Geometrie *3*

4. Analiză Comparativă: Simplex Primal (ASP) vs. Simplex Dual (ASD) *3*

5. Argumente Pro și Contra: Utilizarea în Cercetarea Operațională *4*

SCOPUL PROIECTULUI 4

FORMA GENERALĂ **4**

FORMA STANDARD A UNEI PLS **5**

FUNDAMENTAREA TEORETICĂ A PROBLEMEI LINIARE **6**

1. ALGORITMUL SIMPLEX PRIMAL (ASP) 6

2. ANALIZA DEGENERĂRII ȘI STABILITATEA ALGORITMULUI 7

2.1 Regula lui Bland (Soluția Anti-Ciclu) *8*

TEOREMELE FUNDAMENTALE **8**

CRITERII DE OPTIMALITATE: 8

1. Indicatorii de control (Δj) *8*

2. Criteriul de optim pentru MAXIMIZARE *8*

3. Criteriul de optim pentru MINIMIZARE *9*

CONDIȚIILE DE VERIFICARE: 9

Condiția 1: Fezabilitatea soluției: *9*

Condiția 2: Identitatea valorii optime: *9*

Condiția 3: Validarea structurală prin matrice *9*

PSEUDOCOD ALGORITM SIMPLEX PRIMAL (ASP) **10**

APLICAȚII (REZOLVARE CU PYTHON) **14**

1.PROBLEMA DIETEI 14

2. PROBLEMA TRANSPORTULUI 15

CONCLUZIE **16**

BIBLIOGRAFIE **17**

Introducere

Cercetarea Operațională și Algoritmul Simplex

1. Geneza Cercetării Operaționale (CO)

Cercetarea Operațională s-a constituit ca disciplină științifică autonomă în perioada 1937–1945, fiind un produs direct al necesităților strategice ale Aliaților (coaliția formată în principal din Regatul Unit, SUA și URSS) în timpul celui de-al Doilea Război Mondial.

2. George Dantzig: „Creierul" din spatele Programării Liniare

Dacă Cercetarea Operațională a fost cadrul general, George Dantzig a fost cel care i-a oferit "motorul" computațional în 1947, prin inventarea Algoritmului Simplex.

O Legendă Academică: Înainte de a revoluționa logistica, Dantzig a intrat în istoria matematicii printr-o întâmplare remarcabilă. Student la Berkeley fiind, a întârziat la cursul profesorului Jerzy Neyman. Văzând două probleme pe tablă, le-a notat crezând că sunt tema pentru acasă. S-a chinuit câteva zile să le rezolve, considerându-le "puțin mai dificile decât de obicei". În realitate, acelea erau două teoreme celebre de statistică pe care nimeni nu reușise să le demonstreze până atunci.

3. Mecanismul Simplex: Logică și Geometrie

Algoritmul Simplex este o procedură iterativă folosită pentru a găsi cea mai bună soluție (maximă sau minimă) într-un model matematic liniar.

A. Principiul de funcționare

Matematic, restricțiile unei probleme (ex: buget limitat, timp limitat) definesc o figură geometrică multidimensională numită politop convex. Dantzig a demonstrat că soluția optimă se află întotdeauna într-un vârf (vertex) al acestui politop.

B. Variabilele de Decalaj (Slack Variables)

Pentru ca inegalitățile din viața reală să poată fi procesate prin algebra matricială, ele trebuie transformate în egalități.

4. Analiză Comparativă: Simplex Primal (ASP) vs. Simplex Dual (ASD)

Caracteristică	Simplex Primal (ASP)	Simplex Dual (ASD)
Punct de plecare	Începe dintr-o soluție fezabilă	Începe cu o soluție optimă, dar nefezabilă
Direcția de calcul	Se mișcă spre optim menținând fezabilitatea.	Se mișcă spre fezabilitate menținând optimul.

5. Argumente Pro și Contra: Utilizarea în Cercetarea Operațională

Argumente PRO:

- Eficiență în practică: Deși teoretic poate avea o viteză exponențială (cazul Klee-Minty), în 99% din aplicațiile industriale este incredibil de rapid.
- Analiza de Senzitivitate: Oferă informații despre "Prețurile Umbră" (Shadow Prices) – arată managerilor cât ar fi dispuși să plătească pentru o unitate suplimentară de resursă.

Argumente CONTRA:

- Degenerarea: În anumite matrici complexe, algoritmul poate "cicla" (se învârte între aceleași soluții fără a avansa), necesitând reguli matematice suplimentare pentru corecție.

Scopul proiectului

Scopul acestui proiect îl reprezintă studiul și aplicarea metodelor de programare liniară, cu accent pe algoritmul Simplex Primal, în vederea rezolvării unor probleme de optimizare. Se urmărește înțelegerea fundamentelor teoretice ale metodei, precum și implementarea acesteia într-un limbaj de programare (Python), pentru a evidenția modul în care modelele matematice pot fi utilizate în optimizarea resurselor în situații reale.

<u>Obiectivele proiectului</u>

- definirea și înțelegerea formei generale și a formei standard a problemelor de programare liniară;
- studierea algoritmului Simplex Primal și a mecanismului său de funcționare;
- implementarea algoritmului într-un program Python;
- testarea algoritmului pe exemple concrete;
- analiza rezultatelor și interpretarea lor în context practic.

Forma generală

În forma sa generală, o problemă de programare liniară urmărește identificarea valorii optime a funcției obiectiv $f(x_1, x_2, ..., x_n)$ pe baza unui ansamblu de variabile și supusă unui sistem de constrângeri.

$$opt\ f(x1, x2, \ldots, xn)$$

Optimul unei funcții reprezintă determinarea valorii sale maxime sau minime, alegerea între maximizare și minimizare depinzând de natura problemei.

Maximizarea este urmărită atunci când funcția f exprimă un beneficiu, iar minimizarea este utilizată atunci când f reprezintă un cost sau o cheltuială.

Mulțimea variabilelor $x_1, x_2, ..., x_n$ care definesc funcția f constituie baza procesului de optimizare a acesteia.

Vectorul $x = (x_1, x_2, ..., x_n)$ parcurge o mulțime $X \subset R^n$, mulțime care satisface un anumit număr de condiții. Aceste condiții se numesc restricții, unele fiind speciale, iar altele

suplimentare. Restricțiile suplimentare pot impune diferite tipuri de variabile, precum variabile booleene, întregi sau aleatoare.

$$g_i(x_1, \dots, x_n) \begin{Bmatrix} \leq \\ = \\ \geq \end{Bmatrix} 0 \, , i = \overline{(1, m)}$$

Forma generală și forma matriceală a unei probleme de programare liniară sunt echivalente.

Problema de programare liniară în forma generală:

$$f(x) \; = \; c1x1 \; + \; c2x2 + \dots \; + cnxn$$

$$\sum_{j=1}^{n} a_{ij} x_j \begin{pmatrix} \leq \\ = \\ \geq \end{pmatrix} b_i, \qquad i = \overline{(1, m)}$$

$$x_j \begin{matrix} \geq 0 \\ \leq 0 \\ \in R \end{matrix} \; , \; (\forall) j = \overline{(1, n)}$$

a_{ij} – coeficienții din restricții

b_i – termenii liberi (capacități)

c_j – coeficienții funcției obiectiv (costuri sau beneficii)

x_j – variabile de decizie

Problema de programare liniară în forma matriceală:

$$opt \, f(x) = cx$$

$$A \cdot X \begin{pmatrix} \leq \\ = \\ \geq \end{pmatrix} b$$

$$x_j \begin{matrix} \geq 0 \\ \leq 0 \\ \in R \end{matrix} \; , \; (\forall) j = \overline{(1, n)}$$

$$c = (c_1, \dots, c_2); \qquad A = \begin{pmatrix} a_{11} & a_{12} & \dots & a1n \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{pmatrix}; \qquad x = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix}; \qquad b = \begin{pmatrix} b_1 \\ \vdots \\ b_m \end{pmatrix};$$

Forma standard a unei PLS

Forma standard a unei probleme de programare liniară se definește printr-o structură unitară în care funcția obiectiv este optimizată, toate variabilele de decizie sunt non-negative ($x_j \geq 0$), iar toate restricțiile sunt exprimate sub formă de ecuații.

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2 \\ \dots \\ a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n = b_m \end{cases} \qquad x_1 \geq 0, \; x_2 \geq 0, \; \dots, \; x_n \geq 0$$

Orice problemă de programare liniară poate fi adusă la forma standard prin aplicarea unor reguli matematice specifice, în funcție de natura restricțiilor și a variabilelor. Principalele cazuri de transformare sunt:

Cazul funcției obiectiv: Dacă problema inițială cere maximizarea funcției, aceasta se transformă în minimizarea opusei sale, folosind relația:

$$max\, f(x) \;=\; -\,min\, f(-x)$$

$$\Rightarrow\; Opt\, f(x) \;= (-c1)x1 \;+\; (-c2)x2 + \ldots + (-cn)xn$$

Cazul restricțiilor de tip inegalitate:

Inegalitățile de tip „≤" se transformă în ecuații prin adăugarea unei variabile de compensare non-negative ($y_i \geq 0$).

$$\Rightarrow\; a_{i1}x_1 + a_{i2}x_2 + \cdots + a_{in}x_n + y_i = b_i$$

Inegalitățile de tip „≥" se transformă în ecuații prin scăderea unei variabile de surplus non-negative ($y_i \geq 0$).

$$\Rightarrow\; a_{i1}x_1 + a_{i2}x_2 + \cdots + a_{in}x_n - y_i = b_i$$

Introducerea variabilelor de compensare (numite și variabile ecart) are rolul de a transforma inegalitățile în egalități. Astfel, o variabilă de compensare y_i apare în funcția obiectiv sub forma

$$f(x) = \cdots + 0 \cdot y_i + \cdots.$$

- Cazul variabilelor cu semn restricționat sau liber:
  - Dacă o variabilă trebuie să fie negativă ($x_i \leq 0$), ea se înlocuiește cu $-x_i'$, unde $x_i' \geq 0$.
  - Dacă o variabilă este „liberă" (poate avea orice semn), ea se înlocuiește cu diferența a două variabile non-negative:

$$x_i' \geq 0\,, \qquad -\,x_i'' > 0$$

$$x_i = x_i' - x_i''.$$

În situația în care o variabilă $x_i \in [a, b]$, aceasta poate fi tratată prin introducerea a două restricții suplimentare: $x_i \geq a$ și $x_i \leq b$. Ulterior, aceste restricții sunt prelucrate conform procedurii utilizate în cazul general al inegalităților, transformându-se în egalități prin introducerea variabilelor de compensare corespunzătoare.

Fundamentarea Teoretică a problemei liniare

1. Algoritmul simplex primal (ASP)

Definiție: Algoritmul Simplex este o metodă clasică de rezolvare a programelor liniare, introdusă de Dantzig în 1947.

Este o procedură iterativă, similară în anumite privințe cu eliminarea gaussiană, care transformă succesiv o „formă standard" în alta până la atingerea optimului.

O problemă de programare liniară (în forma generală sau canonică) trebuie adusă la forma standard, astfel încât constrângerile de tip inegalitate (≤) devin egalități (=) prin introducerea unor variabile de compensare (slack variables).

Soluția de Bază Fezabilă:

La fiecare pas, se alege o variabilă care intră în bază (pentru a crește valoarea funcției obiectiv) și una care părăsește baza, folosind regula raportului minim pentru a păstra soluția fezabilă. Astfel, variabilele se împart în:

Variabilele din bază (B): Un set de m variabile (unde m este numărul de constrângeri) ale căror valori sunt determinate în mod unic de sistemul de ecuații.

Variabilele din afara bazei (N): Restul variabilelor, cărora li se atribuie valoarea 0 la fiecare pas.

Echivalența Formelor:

Fiecare iterație rescrie problema într-o formă slack echivalentă, astfel încât valoarea funcției obiectiv să fie vizibilă în mod direct prin termenul constant al ecuației obiectiv.

Procesul continuă prin efectuarea operației de **pivotare.** Iterațiile se repetă până la atingerea unei stări în care nu mai există posibilitatea de îmbunătățire a funcției obiectiv, ceea ce indică atingerea optimului.

Selecția variabilei care intră (Entering Variable):

Se alege o variabilă din afara bazei care are un coeficient pozitiv în funcția obiectiv (pentru probleme de maximizare) criteriul $max\{\Delta j > 0\}$. La problemele de minimizare, criteriul este $max\{\Delta j < 0\}$.

Selecția variabilei care iese (Leaving Variable):

Pentru a menține fezabilitatea, se aplică testul raportului: $min\,\{\frac{x_{Bi}}{a_{ip}}\} \mid aip > 0\}$. Aceasta identifică variabila din bază care ajunge prima la valoarea zero pe măsură ce variabila care intră este crescută.

Transformarea Tabelului:

Elementul aflat la intersecția coloanei variabilei care intră cu linia variabilei care iese devine pivotul. Recalcularea întregului sistem se face conform „Regulii Dreptunghiului" sau prin împărțirea liniei pivotului la valoarea acestuia, transformând coloana pivotului într-o coloană a matricei identitate.

2. Analiza Degenerării și Stabilitatea Algoritmului

Soluții Nedegenerate: Sunt situațiile în care fiecare pivotare conduce la o creștere strictă a funcției obiectiv, garantând terminarea algoritmului într-un număr finit de pași.

Soluții Degenerate: Apar atunci când una sau mai multe variabile din bază au valoarea zero. În acest scenariu, o pivotare poate schimba compoziția bazei fără a modifica valoarea funcției obiectiv.

Ciclarea (Cycling): Este riscul teoretic ca algoritmul să treacă printr-o succesiune de baze fără a progresa, revenind la o bază anterioară.

Ciclarea apare exclusiv în prezența degenerării. Când efectuăm pivotarea:

- Variabila intră în bază, dar, ea nu poate crește deloc (rămâne la valoarea de prag 0)
- Valoarea funcției obiectiv nu se modifică.
- Structura bazei se schimbă (componența setului B), dar soluția rămâne identică în spațiul geometric (rămânem blocați în același vârf al poliedrului).

Dacă acest proces continuă fără o regulă de control, algoritmul poate trece printr-o serie de baze diferite ($B_1 \rightarrow B_2 \rightarrow ... \rightarrow B_k$) și, în final, să revină la baza inițială B_1, intrând într-o buclă infinită.

2.1 Regula lui Bland (Soluția Anti-Ciclu)

Aceasta elimină ambiguitatea atunci când avem mai multe opțiuni pentru pivotare, impunând o ordine strictă bazată pe indicii variabilelor ($x_1, x_2, ..., x_k$):

- **La alegerea variabilei care intră**: Dacă există mai multe variabile cu coeficienți favorabili în funcția obiectiv (ex: mai mulți $\Delta_j>0$), se alege variabila cu cel mai mic indice j.
- **La alegerea variabilei care iese:** Dacă testul raportului minim min $\{\frac{x_B}{a_{ip}}\}$ produce o egalitate, se alege din nou variabila cu cel mai mic indice.

Această regulă garantează matematic că nicio bază nu va fi repetată, forțând algoritmul să iasă din zona de degenerare și să termine execuția.

Teoremele fundamentale

Criterii de optimalitate:

„Testul de optimalitate" este pasul decisiv care arată dacă soluția de bază curentă (vârful în care ne aflăm) este cea mai bună posibilă sau dacă mai trebuie să continuăm pivotarea.

1.Indicatorii de control (Δ_j)

Pentru a efectua testul de optimalitate, se utilizează valorile numite diferențe de selecție (sau costuri reduse), notate de regulă cu Δ_j. Acestea se calculează pentru fiecare variabilă care nu se află în bază (variabilele secundare).

Formula teoretică este $\Delta_j = c_j - z_j$, unde c_j sunt coeficienții din funcția obiectiv, iar z_j este produsul dintre coeficienții variabilelor din bază și coloana variabilei j

În tabelul Simplex, acești indicatori se regăsesc pe ultimul rând (rândul funcției obiectiv).

2. Criteriul de optim pentru MAXIMIZARE

O soluție de bază este optimă pentru o problemă de maxim dacă:

Toate diferențele Δ_j sunt ≤ 0 pentru variabilele din afara bazei.

- Dacă toți $\Delta_j \leq 0$ înseamnă că introducerea oricărei variabile secundare în bază ar duce fie la scăderea valorii funcției obiectiv, fie la menținerea ei (în cazul $\Delta_j = 0$). Prin urmare, valoarea actuală este maximul global.

3. Criteriul de optim pentru MINIMIZARE

Analog, pentru o problemă de minim, soluția este optimă dacă:

Toate diferențele Δ_j sunt ≥0

- Orice variabilă am încerca să aducem în bază, aceasta ar „scumpi" soluția (ar crește valoarea funcției), deci am ajuns deja la cel mai mic cost posibil.

Cazul de „Optim Infinit" (Nemărginire)

Testul de optimalitate poate eșua într-un mod specific care oprește algoritmul fără a găsi o soluție finită:

Dacă există o variabilă x_j care ar trebui să intre în bază (conform criteriului de intrare în bază), dar toate elementele de pe coloana ei sunt ≤0 ($a_{ij} \leq 0$), atunci testul raportului minim nu poate fi efectuat. Concluzia este că funcția obiectiv este nemărginită pe mulțimea soluțiilor fezabile (optimul este ∞).

Condițiile de verificare:

Condiția 1: Fezabilitatea soluției:

Această condiție verifică dacă soluția extrasă din ultimul tabel respectă restricția de nenegativitate a programării liniare.

Notație: $\forall x \in X_0, x \geq 0$

Explicație: Se verifică toate componentele vectorului soluție X_0. Valorile extrase din coloana termenilor liberi a tabelului I_{STOP} trebuie să fie pozitive sau nule. Dacă există vreo valoare negativă, înseamnă că algoritmul a ieșit din domeniul de fezabilitate din cauza unei erori de calcul.

Condiția 2: Identitatea valorii optime:

Această condiție validează valoarea funcției obiectiv, comparând rezultatul din tabel cu calculul direct pe modelul inițial.

Notație: $f(opt) = f_{optim}$, unde $X_{optim} = X_B(I_{STOP}), iar\ I_{STOP} = Z^k(I_{STOP})$

Explicație:

- f_{opt} este valoarea citită direct din celula funcției obiectiv din ultimul tabel (I_{STOP})
- f(opt) este valoarea calculată manual prin înlocuirea variabilelor de bază finale (X_b din iterația de stop) în funcția obiectiv inițială: $f(X_{opt}) = \sum c_j \cdot x_j$
- Cele două trebuie să coincidă perfect.

Condiția 3: Validarea structurală prin matrice

Aceasta este cea mai riguroasă probă, care verifică legătura dintre baza inițială și baza finală folosind matricea de transformare (matricea de structură).

Notație: $\boldsymbol{X_B(I_0) = S \cdot X_B(I_{STOP})}$.

Explicație:

- **$X_B(I_0)$** reprezintă vectorul variabilelor de bază din primul tabel (iterația I_0), adică termenii liberi inițiali (b).
- **$X_B(I_{STOP})$** reprezintă vectorul variabilelor de bază din ultimul tabel.
- **S** este matricea formată din coloanele tabelului inițial (matricea A originală) care corespund variabilelor de bază din ultimul tabel.
- **Regula**: Dacă se înmulțește matricea S cu valorile de bază finale, trebuie să se obțină exact valorile termenilor liberi de la care s-a plecat. Aceasta demonstrează că toate operațiile de pivotare (echivalente cu înmulțirea cu inversa bazei) au fost executate corect.

Pseudocod Algoritm Simplex Primal (ASP)

```
// DATE DE INTRARE
CITEȘTE A, b, c
CITEȘTE Semne, Condiții_x, Tip_Opt, M

// PASUL 1: VERIFICAREA TERMENILOR LIBERI
PENTRU i ← 1 .. m EXECUTĂ
   DACĂ b[i] < 0 ATUNCI
      b[i] ← -b[i]
      A[i] ← -A[i]
      DACĂ Semne[i] = "<=" ATUNCI
         Semne[i] ← ">="
      ALTFEL DACĂ Semne[i] = ">=" ATUNCI
         Semne[i] ← "<="
      SFÂRȘIT DACĂ
   SFÂRȘIT DACĂ
SFÂRȘIT PENTRU
b_init_std ← b      // Salvare pentru validarea V3

//  PASUL 2: STANDARDIZARE (PL)=>(PLS)
// Regula R1: x>=0 în COND 3
PENTRU j ← 1 .. n_init EXECUTĂ
```

DACĂ Condiții_x[j] = "<=0" **ATUNCI**

Substituție x[j] = -x'[j]

// se înmulțește coloana j din A și coeficientul c[j] cu -1

ALTFEL DACĂ Condiții_x[j] = "liber" **ATUNCI**

Substituție x[j] = x'[j] - x"[j]

// se adaugă două coloane noi în A, doi coeficienți în c, se actualizează totalul de variabile

SFÂRȘIT DACĂ

SFÂRȘIT PENTRU

// Regula R2: Transformarea restricțiilor în egalități în COND 2

PENTRU i ← 1 .. m **EXECUTĂ**

DACĂ Semne[i] = "<=" **ATUNCI**

Adaugă variabila de ecart y[i] în A

Cj[j] ← 0

Baza[i] ← indice(y[i])

ALTFEL DACĂ Semne[i] = ">=" **ATUNCI**

Adaugă variabila de surplus y[i] (coef 0)

Adaugă variabila artificială z[i] în A

Cj[j_y] ← 0

Cj[j_z] ← (Tip_Opt = "MIN" ? M : -M)

Baza[i] ← indice(z[i])

ALTFEL DACĂ Semne[i] = "=" **ATUNCI**

Adaugă variabila artificială z[i] în A

Cj[j_z] ← (Tip_Opt = "MIN" ? M : -M)

Baza[i] ← indice(z[i])

SFÂRȘIT DACĂ

SFÂRȘIT PENTRU

A_T0 ← A *// Matricea extinsă inițială pentru validarea V3 (S)*

// <u>PASUL 3: TABEL SIMPLEX</u>

Optim_Gasit ← FALS

Pas ← 0

```
CÂT TIMP (Optim_Gasit = FALS) ȘI (Pas < 1000) EXECUTĂ
  // Calcul CB, Z, Delta
  CB ← coeficienții Cj ai variabilelor aflate curent în Bază
  Z ← Σ (CB[i] · b[i])
  PENTRU j ← 1 .. n_total EXECUTĂ
    Zj ← Σ (CB[i] · A[i][j])
    Delta[j] ← Cj[j] - Zj
  SFÂRȘIT PENTRU
  // AFIȘEAZĂ TABELUL SIMPLEX CURENT (T[Pas])
  // Tabelul conține: CB, Baza, XB, a_1... a_n și rândul Delta
  AFIȘEAZĂ Tabel_Simplex(Pas, CB, Baza, b, A, Z, Delta)
  // Test de optimalitate
  DACĂ Tip_Opt = "MAX" ATUNCI
    DACĂ toate Delta[j] ≤ 0 ATUNCI
      Optim_Gasit ← ADEVĂRAT
    SFÂRȘIT DACĂ
  ALTFEL   // MIN
    DACĂ toate Delta[j] ≥ 0 ATUNCI
      Optim_Gasit ← ADEVĂRAT
    SFÂRȘIT DACĂ
  SFÂRȘIT DACĂ
  DACĂ Optim_Gasit = FALS ATUNCI
    // Alegerea elementului pivot (k, r)
    DACĂ Tip_Opt = "MAX" ATUNCI
      k ← coloana cu Delta[j] pozitiv maxim
    ALTFEL
      k ← coloana cu Delta[j] negativ minim
    SFÂRȘIT DACĂ
    // Calcul rapoarte (criteriul theta)
    PENTRU i ← 1 .. m EXECUTĂ
      DACĂ A[i][k] > 0 ATUNCI
        Rapoarte[i] ← b[i] / A[i][k]
```

```
        SFÂRȘIT DACĂ
      SFÂRȘIT PENTRU
      r ← indicele raportului minim pozitiv
      DACĂ nu există rapoarte pozitive ATUNCI
        AFIȘEAZĂ "Optim nemărginit"
        IEȘIRE ALGORITM
      SFÂRȘIT DACĂ

      // Pivotare Gauss–Jordan
      Pivot ← A[r][k]
      A[r] ← A[r] / Pivot
      b[r] ← b[r] / Pivot
      PENTRU i ← 1 .. m, i ≠ r EXECUTĂ
        Factor ← A[i][k]
        A[i] ← A[i] - Factor · A[r]
        b[i] ← b[i] - Factor · b[r]
      SFÂRȘIT PENTRU
      Baza[r] ← k
      Pas ← Pas + 1
    SFÂRȘIT DACĂ
  SFÂRȘIT CÂT TIMP

  // PASUL 4: VALIDAREA SOLUȚIEI
  AFIȘEAZĂ "Validare V1 (Nenegativitate):"
  Verifică xj* ≥ 0
  AFIȘEAZĂ "Validare V2 (Verificare f):"
  x_orig ← Reconstrucție(x', x")
  Verifică Σ (c[j] · x_orig[j]) = Z
  AFIȘEAZĂ "Validare V3 (Relația S):"
  S ← coloanele bazei din A_T0
  Verifică S · b_final = b_init_std
SFÂRȘIT ALGORITM
```

Aplicații (Rezolvare cu Python)

https://github.com/dariailiescu15/Algoritm-Simplex/blob/main/Proiect_Program_ASP.ipynb

1.Problema Dietei

O persoană vrea să țină o dietă, dar are un buget limitat, așa că vrea să cheltuiască cât mai puțin. Cerințele nutriționale sunt următoarele: 2000 kcal, 55 g proteine, 800 mg calciu. Se consideră următorul plan alimentar:

Mâncare	Cantitate	Calorii (kcal)	Proteine (g)	Calciu (mg)	Preț ($)
Iaurt	28g	110	4	2	0.30
Vită	100g	205	32	12	2.40
Ouă fierte	2 (dim. L)	160	13	54	1.30
Lapte	237ml	160	8	285	0.90
Plăcintă	170g	420	4	22	2.00
Piept de pui	260g	260	14	80	1.90

Putem reprezenta numărul de porții al fiecărui tip de mâncare din plan considerând variabilele:

x1 pentru iaurt x3 pentru ouă fierte x5 pentru plăcintă

x2 pentru vită x4 pentru lapte x6 pentru piept de pui

Formularea problemei de programare liniară

1) $MIN_x f(x) = 0.3x_1+2.4x_2+1.3x_3+0.9x_4+2x_5+1.9x_6$

2) $\begin{cases} 110x_1 + 205x_2 + 160x_3 + 160x_4 + 420x_5 + 260x_6 \geq 2000 \\ 4x_1 + 32x_2 + 13x_3 + 8x_4 + 4x_5 + 14x_6 \geq 55 \\ 2x_1 + 12x_2 + 54x_3 + 285x_4 + 22x_5 + 80x_6 \geq 800 \end{cases}$

3) $x_{1-6} \geq 0$

Știm că la final vom obține soluția 6.71($) corespunzătoare meniului: $\begin{pmatrix} 14.24\ x_1 \\ 0\ x_2 \\ 0\ x_3 \\ 2.71\ x_4 \\ 0\ x_5 \\ 0\ x_6 \end{pmatrix}$.

Python a realizat iterații de la I_0 până la I_5 , unde a reușit în final să verifice criteriul de optim și să valideze soluția.

Rezultat: $f_{min}(PL) = f_{min}(PLS) = \frac{208}{31} \sim 6.71$ respectiv x_{min} (PL) = $\begin{pmatrix} x_1 = \frac{1225}{86} \sim 14.24 \\ x_2 = 0 \\ x_3 = 0 \\ x_4 = \frac{268}{99} \sim 2.71 \\ x_5 = 0 \\ x_6 = 0 \end{pmatrix}$

2. Problema Transportului

O companie siderurgică produce role de oțel în 3 fabrici (Gary, Cleveland, Pittsburgh) și trebuie să le livreze către 7 clienți (fabrici de automobile). Obiectivul este determinarea celui mai ieftin plan de transport, astfel încât toate comenzile să fie satisfăcute fără a depăși capacitatea de producție a fiecărei fabrici.

Cerințele sunt următoarele: producția totală este de 6900 tone, iar cererea totală a fabricilor este de 6900 tone. Se consideră următorul plan de costuri și cantități:

Fabrica (destinație)	Gary (Cost $/t)	Cleveland (Cost $/t)	Pittsburgh (Cost $/t)	Cerere (tone)
Framingham (FRA)	39	27	24	900
Detroit (DET)	14	9	14	1200
Lansing (LAN)	11	12	17	600
Windsor (WIN)	14	9	13	400
St. Louis (STL)	16	26	28	1700
Fremont (FRE)	82	95	99	1100
Lafayette (LAF)	8	17	20	1000
Capacitate (tone)	**1400**	**2600**	**2900**	**6900**

Putem reprezenta numărul de tone transportate pe fiecare rută considerând variabilele:

$x_1 - x_7$ pentru transportul de la Gary către cele 7 destinații (FRA, DET, LAN, WIN, STL, FRE, LAF)

$x_8 - x_{14}$ pentru transportul de la Cleveland către cele 7 destinații

$x_{15} - x_{21}$ pentru transportul de la Pittsburgh către cele 7 destinații

<u>Formularea problemei de programare liniară</u>

1) $MIN_x f(x) = 39x_1 + 14x_2 + 11x_3 + 14x_4 + 16x_5 + 82x_6 + 8x_7 + 9x_9 + 12x_{10} + 9x_{11} + 26x_{12} + 95x_{13} + 17x_{14} + 24x_{15} + 14x_{16} + 17x_{17} + 13x_{18} + 28x_{19} + 99x_{20} + 20x_{21}$

2) $$\begin{cases} x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 = 1400 \\ x_8 + x_9 + x_{10} + x_{11} + x_{12} + x_{13} + x_{14} = 2600 \\ x_{15} + x_{16} + x_{17} + x_{18} + x_{19} + x_{20} + x_{21} = 2900 \\ x_1 + x_8 + x_{15} = 900 \\ x_2 + x_9 + x_{16} = 1200 \\ x_3 + x_{10} + x_{17} = 600 \\ x_4 + x_{11} + x_{18} = 400 \\ x_5 + x_{12} + x_{19} = 1700 \\ x_6 + x_{13} + x_{20} = 1100 \\ x_7 + x_{14} + x_{21} = 1000 \end{cases}$$

3) $x_{1-21} \geq 0$

Rezultat așteptat după testare: cost minim: **$196200**

Python a realizat standardizarea adăugând o variabilă artificială (-M) rezultând astfel 31 coeficienți.

Am considerat M de la început 1000, iar programul a realizat iterații de la I_0 până la I_{11} , unde a reușit în final să verifice criteriul de optim și să valideze soluția.

Rezultat : f_{min}(PL) = f_{min}(PLS) = 196200 (rezultat așteptat)

$$X_{min}(PL) = \begin{pmatrix} x_1 = 0 \\ x_2 = 0 \\ x_3 = 0 \\ x_4 = 0 \\ x_5 = 0 \\ x_6 = 1100 \\ x_7 = 300 \\ x_8 = 0 \\ x_9 = 1200 \\ x_{10} = 600 \\ x_{11} = 400 \\ x_{12} = 0 \\ x_{13} = 0 \\ x_{14} = 400 \\ x_{15} = 900 \\ x_{16} = 0 \\ x_{17} = 0 \\ x_{18} = 0 \\ x_{19} = 1700 \\ x_{20} = 0 \\ x_{21} = 300 \end{pmatrix}$$

	Gary	Cleveland	Pittsburgh
Framingham (FRA)	0	0	900
Detroit (DET)	0	1200	0
Lansing (LAN)	0	600	0
Windsor (WIN)	0	400	0
St. Louis (STL)	0	0	1700
Fremont (FRE)	1100	0	0
Lafayette (LAF)	300	400	300

Concluzie

Prezentul proiect a demonstrat rigoarea **Programării Liniare** și eficiența ***Algoritmului Simplex Primal*** în identificarea soluțiilor optime sub constrângeri structurale. Analiza coroborată a fundamentelor teoretice cu rezultatele implementării software conduce la următoarele concluzii:

- **Validarea Modelării**: Procesul de standardizare a reconfirmat necesitatea transformării riguroase a inegalităților în ecuații prin introducerea variabilelor de ecart și surplus.
- **Convergența și Stabilitatea**: Utilizarea *Regulii lui Bland* s-a dovedit esențială pentru garantarea terminării algoritmului într-un număr finit de pași, eliminând riscul de ciclu în prezența soluțiilor degenerate.
- **Performanța Computațională**: Implementarea în Python a atestat eficiența metodei, obținând soluții exacte pentru probleme complexe, precum cea a *Transportului* (cost de *196.200 $* în 12 iterații).
- **Rigoarea Verificării**: Fiabilitatea rezultatelor a fost consolidată prin cele trei protocoale de validare (V1, V2, V3), confirmând nenegativitatea, identitatea valorii optime și integritatea structurală a bazei.

În concluzie, proiectul atestă faptul că ***Algoritmul Simplex*** rămâne un pilon indispensabil în optimizarea sistemelor, oferind o metodologie robustă pentru transformarea limitărilor de resurse în decizii strategice fundamentate matematic.

Bibliografie

- 2026 Curs Cercetări Operaționale, Lect. Dr. Simona Mihaela BIBIC, Facultatea de Științe Aplicate UNSTPB
- ©2004 Editura FAIR PARTNERS ISBN 973-8470-28-5, Vasile Masgras, CERCETĂRI OPERAȚIONALE
- Ing. Radu BIG CERCETĂRI OPERAŢIONALE PENTRU FACULTĂŢI TEHNICE, NOTE DE CURS, BAIA MARE, DECEMBRIE 2004
- FUNDAMENTALS OF MANAGEMENT SCIENCE 1988 FOURTH EDITION, EFRAIM TURBAN & JACK R. MEREDITH
- 2007, Editura FAIR PARTNERS, MATEMATICI APLICATE ÎN ECONOMIE, Tiberiu VLADISLAV
- Dantzig, George Bernard, Linear Programming and Extensions. Santa Monica, CA: RAND Corporation, 1963. https://www.rand.org/pubs/reports/R366.html.
- Copyright © 2003 by Robert Fourer, David M. Gay and Brian W. Kernighan. AMPL: A Modeling Language for Mathematical Programming https://ampl.com/resources/ampl-book/#
- Robert E. Bixby ILOG, Inc. and Rice University Solving Linear and Integer Programs https://co-at-work.zib.de/berlin2009/downloads/2009-09-24/2009-09-24-0900-BB-Linear-Programming-1.pdf
- Randomised Algorithms Lecture7: Linear Programming: Simplex Algorithm, Thomas Sauerwald, University of Cambridge https://www.cl.cam.ac.uk/teaching/2324/RandAlgthm/4up/lec7_simplex.pdf
- https://en.wikipedia.org/wiki/Simplex_algorithm
- https://masarat-sy.org/en/simplex-method/
- https://www.lancaster.ac.uk/stor-i-student-sites/ben-lowery/2022/03/linear-programming-and-the-birth-of-the-simplex-algorithm/
- https://www.informs.org/Explore/History-of-O.R.-Excellence/Bibliographies/The-Origins-of-OR